



Path Planning

V Muthukumar





What is Path Planning

- Path-planning is an important primitive for autonomous mobile robots that lets robots find the shortest path or otherwise optimal path between two points.
- Path-planning requires a map of the environment and the robot to be aware of its location with respect to the map
- What do we cover here
 - Map Representation
 - Planning Algorithms
 - Coverage Problem*.





Map Representation

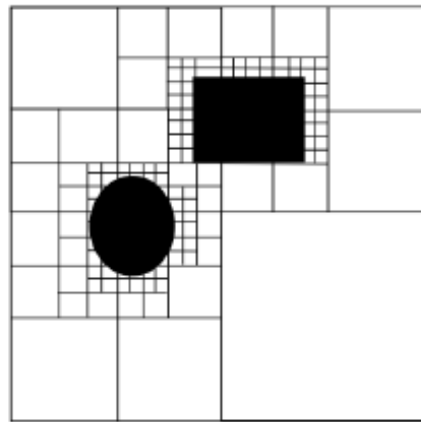
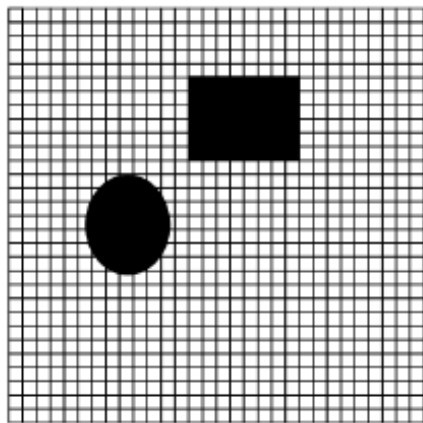
- Discrete (Grid) & Continuous (Topological), also Feature based (landmark)*
- Discrete (Grid)
 - map is sub-divided into chunks of equal (e.g., a grid or hexagonal map) or
- Topological maps
 - differing sizes
 - continuous Representation
- Both requires the definition of inner (obstacles) and outer boundaries, typically in the form of a polygon, whereas paths can be encoded as sequences of points defined by real numbers.
 - Obstacle Maps
 - occupancy grid map.





Grid Maps

- Disadvantages of grid maps are their large memory requirements as well as computational time to traverse data structures with large numbers of vertices.



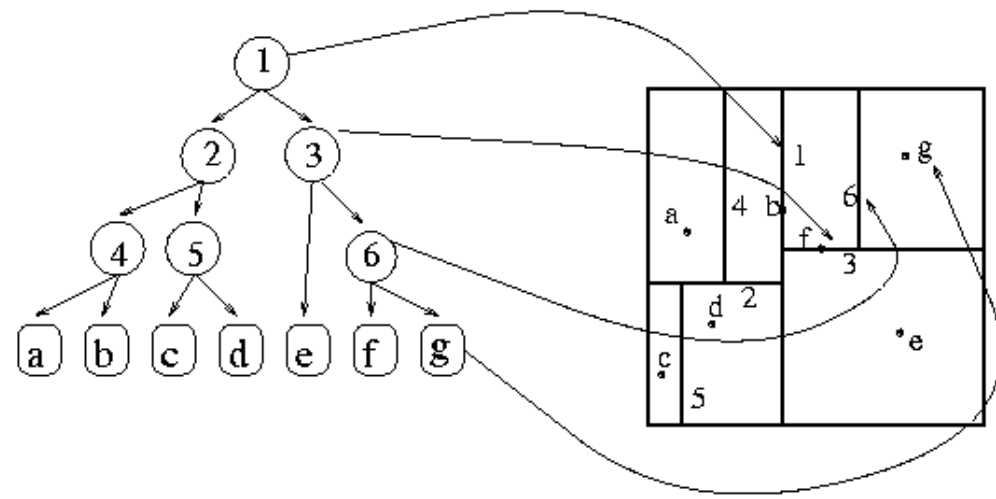
quadtree



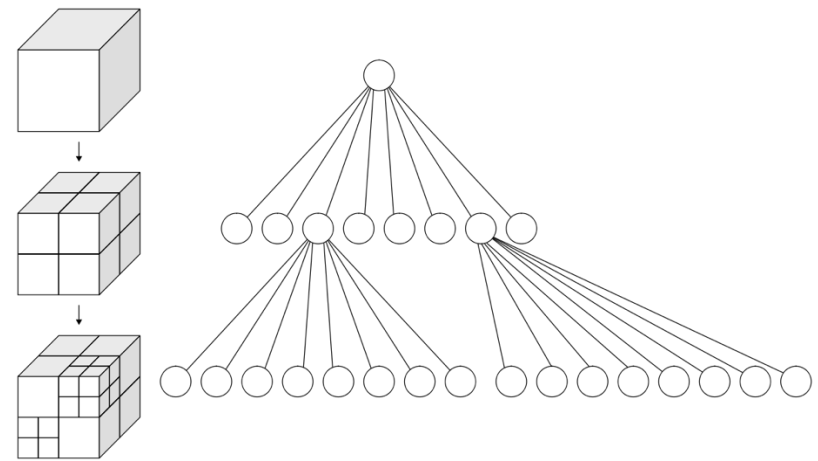


Continuous Representations

- Grid map as k-d tree. k-d tree recursively breaks the environment into k pieces.
- 3D point clouds stored in an 8-d tree, also known as an Octree.



KD - Tree



Octo - Tree

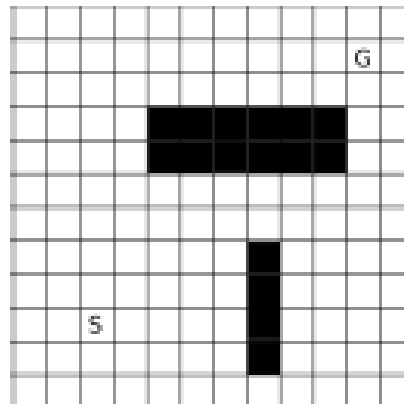
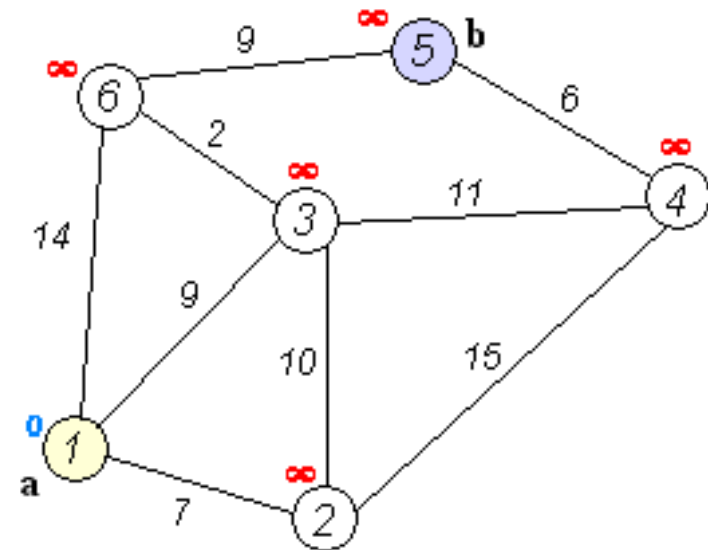
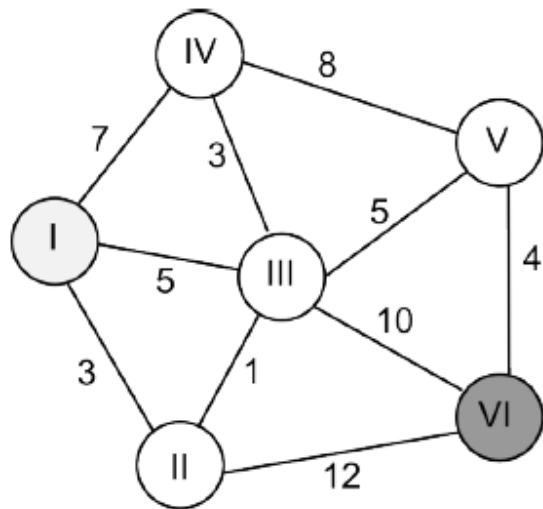




Path-Planning Algorithms for Topology and Grid Maps

- “shortest” path from one vertex to another through a connected graph

A generic path planning problem from vertex I to vertex VI. The shortest path is I-II-III-V-VI with length 13.



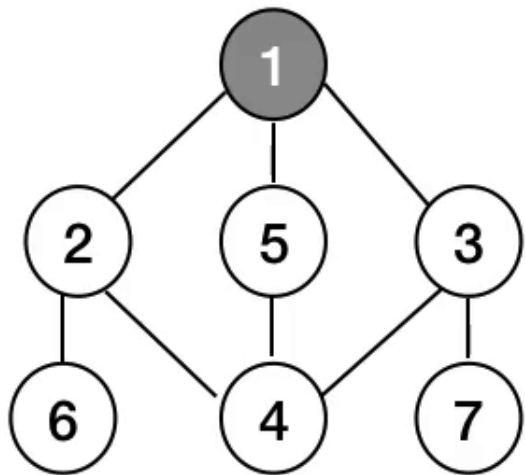
11	10	9	10	11	12									
10	9	8	9	10	11	12								G
9	8	7	8	9	10	11	12							
8	7	6	7											
7	6	5	6											
6	5	4	5	6	7	8	9	10	11	12				
5	4	3	4	5	6	7	8	9	10	11	12			
4	3	2	3	4	5	6			10	11	12			
3	2	1	2	3	4	5			11	12				
2	1	5	1	2	3	4			10	11	12			
1	2	1	2	3	4	5			9	10	11	12		
4	3	2	3	4	5	6	7	8	9	10	11			

11	10	9	10	11	12	13	14	15	16					
10	9	8	9	10	11	12	13	14	15	G				
9	8	7	8	9	10	11	12	13	14	15				
8	7	6	7							14	15			
7	6	5	6							13	14			
6	5	4	5	6	7	8	9	10	11	12	13			
5	4	3	4	5	6	7	8	9	10	11	12			
4	3	2	3	4	5	6			10	11	12	13		
3	2	1	2	3	4	5			11	12	13	14		
2	1	5	1	2	3	4			10	11	12	13		
1	2	1	2	3	4	5			9	10	11	12		
4	3	2	3	4	5	6	7	8	9	10	11			

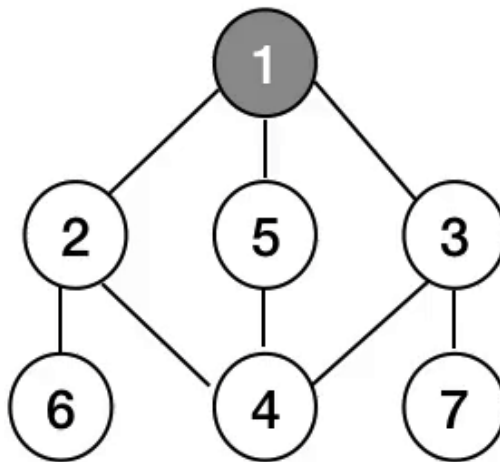




BFS & DFS Algorithms



BFS



DFS





Dijkstra's algorithm

(weighted BFS)

```
function Dijkstra(Graph, source):
```

```
    create vertex set Q
```

```
    for each vertex v in Graph:
```

```
        dist[v] ← INFINITY
```

```
        prev[v] ← UNDEFINED
```

```
        add v to Q
```

```
    dist[source] ← 0
```

```
    while Q is not empty:
```

```
        u ← vertex in Q with min dist[u]
```

```
        remove u from Q
```

```
        for each neighbor v of u:
```

```
            alt ← dist[u] + length(u, v)
```

```
            if alt < dist[v]:
```

```
                dist[v] ← alt
```

```
                prev[v] ← u
```

```
    return dist[], prev[]
```

```
// Init
```

```
// Unk
```

```
// Prev
```

```
// All
```

```
// Dist
```

```
// Node with the least distance
```

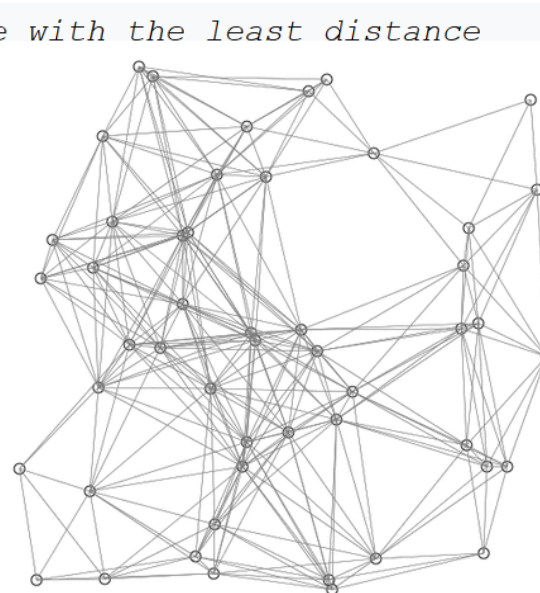
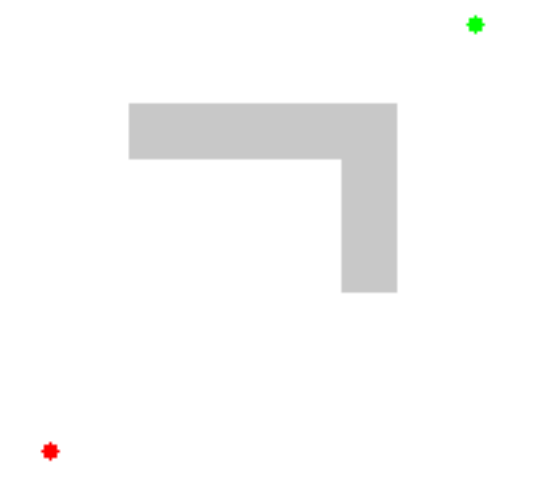
```
// will
```

```
// when
```

```
// A sh
```

Grid based

v
from source
ted nodes)



ind

Continuous Representation

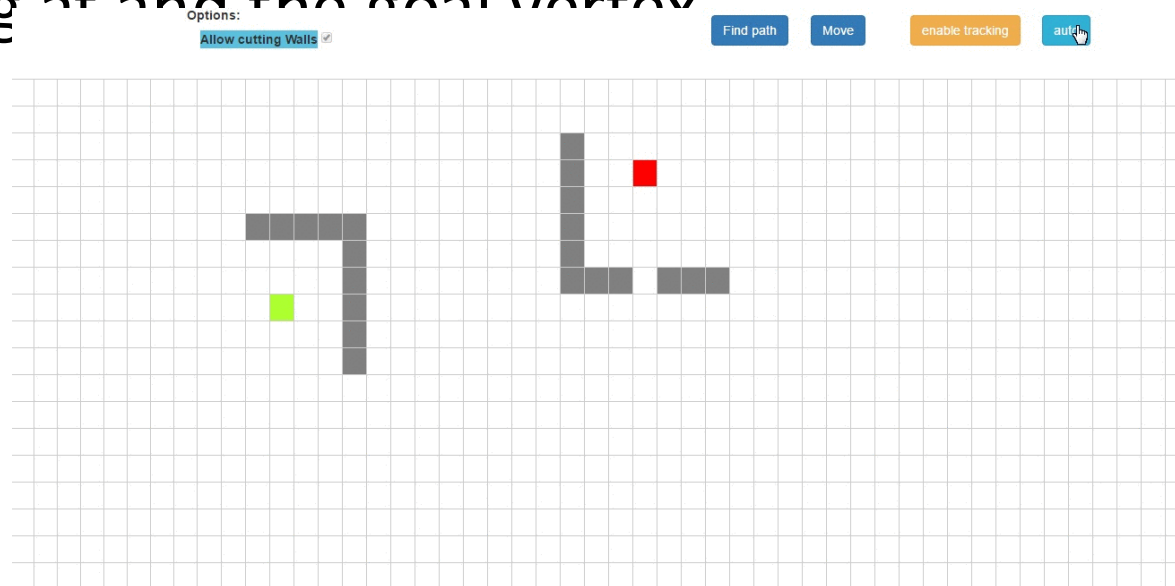




A* Algorithm

(Heuristic DA)

- Instead of exploring in all directions, knowledge of an approximate direction of the goal could help avoiding exploring nodes that are obviously wrong to a human observer.
- estimated cost - Euclidean distance or the Manhattan distance between the vertex we are looking at and the goal vertex



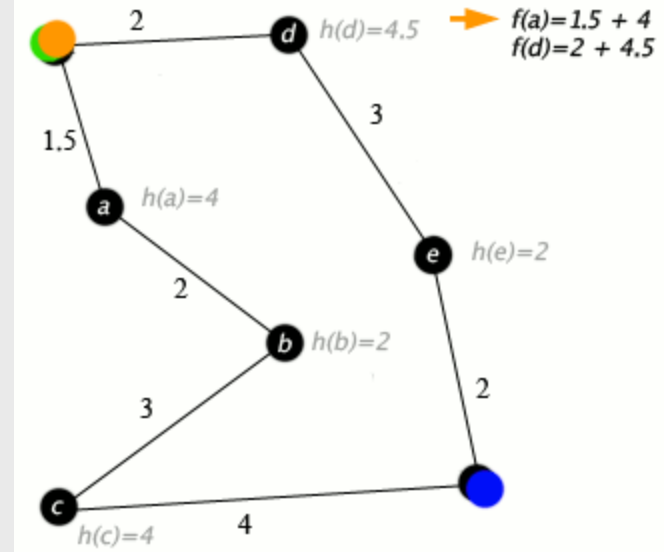


A* Algorithm Pseudocode

```
function A*(start, goal)
    open_list = set containing start
    closed_list = empty set
    start.g = 0
    start.f = start.g + heuristic(start, goal)
    while open_list is not empty
        current = open_list element with lowest f cost
        if current = goal
            return construct_path(goal) // path found
        remove current from open_list
        add current to closed_list
        for each neighbor in neighbors(current)
            if neighbor not in closed_list
                neighbor.f = neighbor.g + heuristic(neighbor, goal)
                if neighbor is not in open_list
                    add neighbor to open_list
                else
                    openneighbor = neighbor in open_list
                    if neighbor.g < openneighbor.g
                        openneighbor.g = neighbor.g
                        openneighbor.parent = neighbor.parent
    return false // no path exists

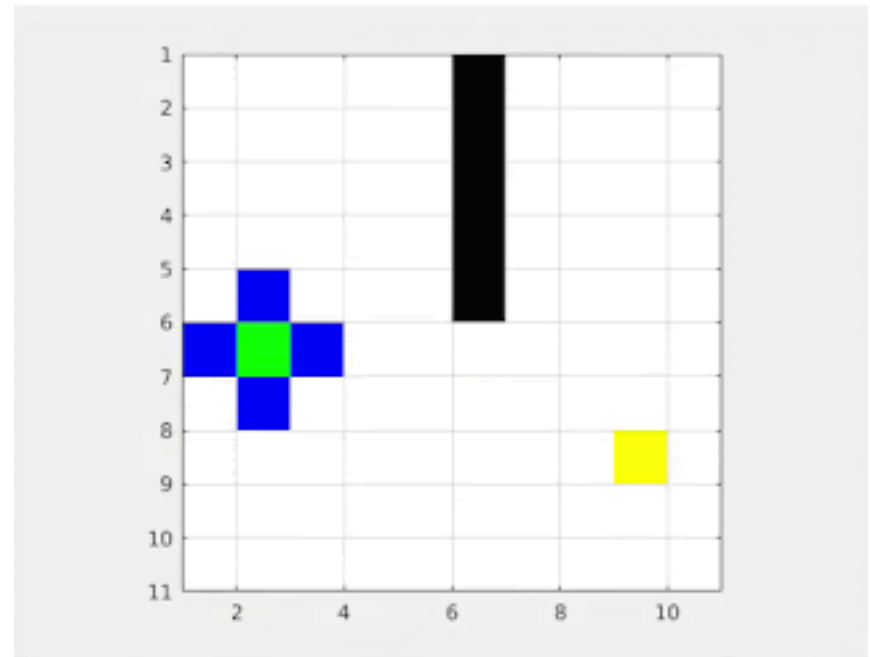
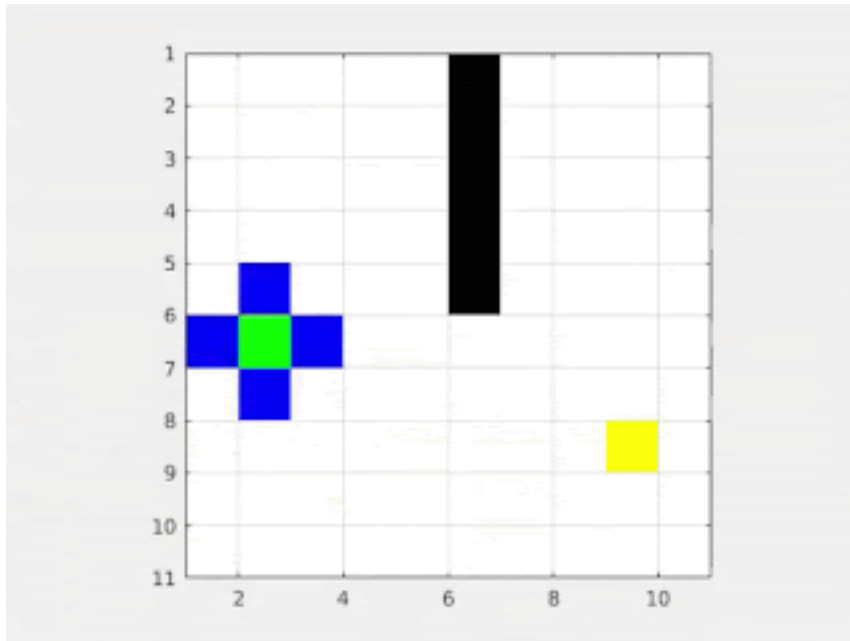
function neighbors(node)
    neighbors = set of valid neighbors to node // check for obstacles here
    for each neighbor in neighbors
        if neighbor is diagonal
            neighbor.g = node.g + diagonal_cost // eg. 1.414 (pythagoras)
        else
            neighbor.g = node.g + normal_cost // eg. 1
        neighbor.parent = node
    return neighbors

function construct_path(node)
    path = set containing node
    while node.parent exists
        node = node.parent
        add node to path
    return path
```





Dijkstra's Algorithm Vs A* Algorithm





Sampling-based Path Planning

- Complete solutions are often unfeasible, complete if the resolution is fine-grained.
- Sampling-based planners create possible paths by randomly adding points to a tree until some solution is found or time expires.
- Sampling-based planner can quickly explore a large portion of space and refines a solution as time goes on.
- Sampling-based path planners are probabilistic complete
- Random Path Planning / Potential-Field Planner, Rapidly-exploring Random Trees (RRT) and Probabilistic Roadmaps (PRM)





Random Path Planning (RPP)

- Also referred to as “Potential-Field Planner”
- An artificial potential field (APF) is a scalar function over the free space, C_{free} .
- The algorithm tries to connect the q_{init} and q_{goal} through two types of motions:
 - down motion and escape motion.
- The down motion is named so because the motion travels towards q_{goal} , decreasing the value along the potential field represented by U ;
 - a function U that maps the object-free configuration space C_{free} to a set of non-negative real number R
- escape motion simply means the motion tries to escape from C_{obs} , attempting to avoid getting stuck in the local minima.





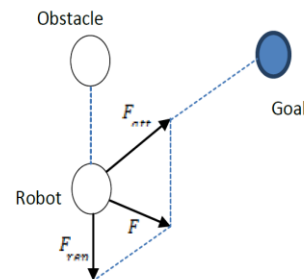
Potential-Field Planner

- APF function can be represented as

$$U(q) = U_{att}(q) + U_{rep}(q) \quad \left\{ \begin{array}{l} U(q) = \text{artificial potential field} \\ U_{att}(q) = \text{attractive field} \\ U_{rep}(q) = \text{repulsive field} \end{array} \right.$$

- The attractive force is the negative gradient of attractive field and the repulsive force is the negative gradient of repulsive field.

$$\begin{aligned} F(q) &= -\nabla U(q) \\ &= -\nabla U_{att}(q) - \nabla U_{rep}(q) \\ F(q) &= F_{att}(q) + F_{rep}(q) \end{aligned} \quad \left\{ \begin{array}{l} F(q) = \text{artificial force,} \\ F_{att}(q) = \text{attractive force,} \\ F_{rep}(q) = \text{repulsive force.} \end{array} \right.$$





Potential-Field Planner

- The attractive field between robot and goal is assembled to drag the robot to the goal area.

$$U_{att}(q) = \frac{1}{2} \times k_a \times (q - q_d)^2$$

$$= \frac{1}{2} k_a \rho_{goal}^2(q)$$

Consider that the Cartesian coordinate of a robot is $q=(x,y)^T$

k_a = positive coefficient of gravity for APF,
 q = current position vector of the robot,
 q_d = current position vector of the target.
 $\rho_{goal}(q) = \|q - q_d\|$

$$F_{att}(q) = -\nabla$$

$$= -\frac{1}{2} k_a \rho_{goal}^2(q)$$

$$= -k_a (q - q_d)$$

$$F_{att-x}(q) = -k_a (x - x_d)$$

$$F_{att-y}(q) = -k_a (y - y_d)$$

$$U_{rep}(q) = \begin{cases} \frac{1}{2} k_b \left(\frac{1}{d(q)} - \frac{1}{d_0} \right)^2, & d(q) \leq d_0 \\ 0, & d(q) \geq d_0 \end{cases}$$

k_b = repulsion gain coefficient,
 d = distance between the robot and the obstacle,
 d_0 = distance of the obstacle repulsive force field.
 $d = \|q - q_c\|$

- The repulsion potential function is

$$F_{rep}(q) = \begin{cases} k_b \left(\frac{1}{d(q)} - \frac{1}{d_0} \right) \left(\frac{1}{d^2(q)} \right) \left(\frac{\partial d(q)}{\partial x} \right), & d(q) \leq d_0 \\ 0, & d(q) \geq d_0 \end{cases}$$

$$F_{rep}(q) = \begin{cases} k_b \left(\frac{1}{d(q)} - \frac{1}{d_0} \right) \left(\frac{1}{d^2(q)} \right) \left(\frac{q - q_c}{\|q - q_c\|} \right), & d(q) \leq d_0 \\ 0, & d(q) \geq d_0 \end{cases}$$

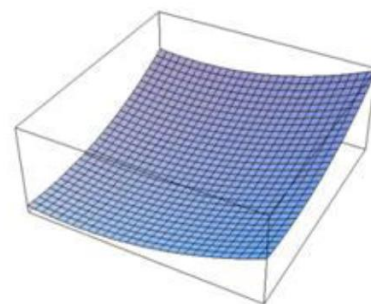
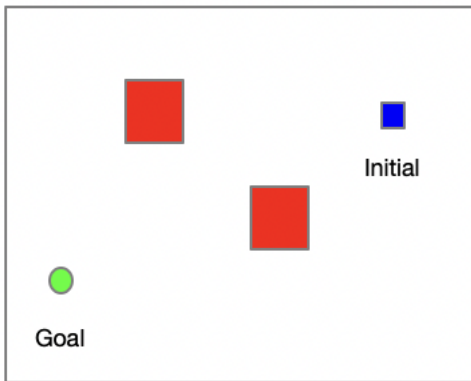
$x - x_c \quad y - y_c$





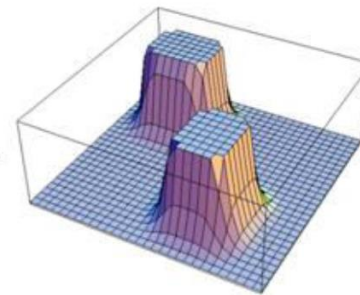
Potential-Field Planner

- To navigate, the robot applies a force proportional to the negated gradient of the potential field.
- Place a regular grid G over the configuration space
- Compute the potential field over G
- Search G using a best-first algorithm with potential field as the heuristic function



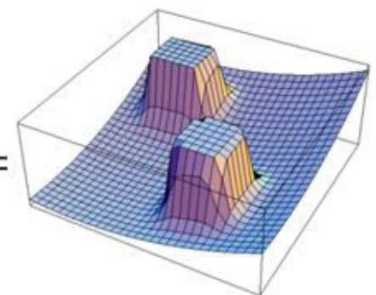
Attractive Potential

+



Repulsive Potential

=



Whole Potential



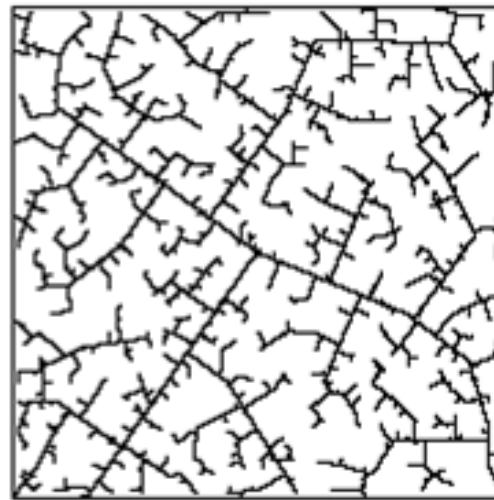


Rapidly-exploring Random Trees

- RRT can be understood as growing a single tree from a robot's starting point until one of its branches hit a goal.
- RRT's are known as single-query path-planning algorithms



45 iterations



390 iterations





RRT

- Rapidly Exploring Random Tree (RRT) searches for a path from the initial configuration to the goal configuration by expand a search tree.
- Start building a tree starting from the position of the robot.
- A point in the space is randomly sampled, it is checked if that point collides with an obstacle in the space. If there is a collision, this point is thrown out.
- If there exists a straight line path between the sampled point and the existing point, the sampled point is added to the tree with the existing point as its parent node.
- Each time a node is added to the tree and is less than some threshold distance from the goal position, check if the goal is directly reachable by a straight line. If so terminate the algorithm
- If the goal position is unreachable, additional points are sampled. The process is repeated until the goal is reached.
- Determine the shortest path from the goal to the initial position. Path smoothing can also be applied to the shortest path.

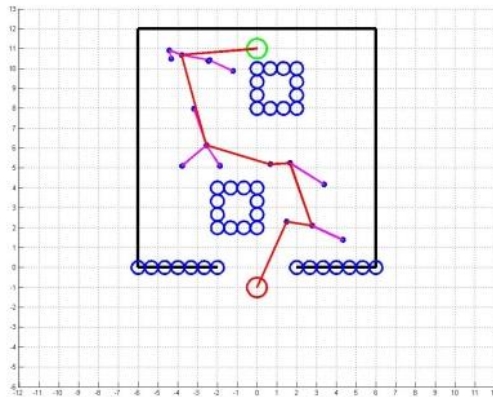
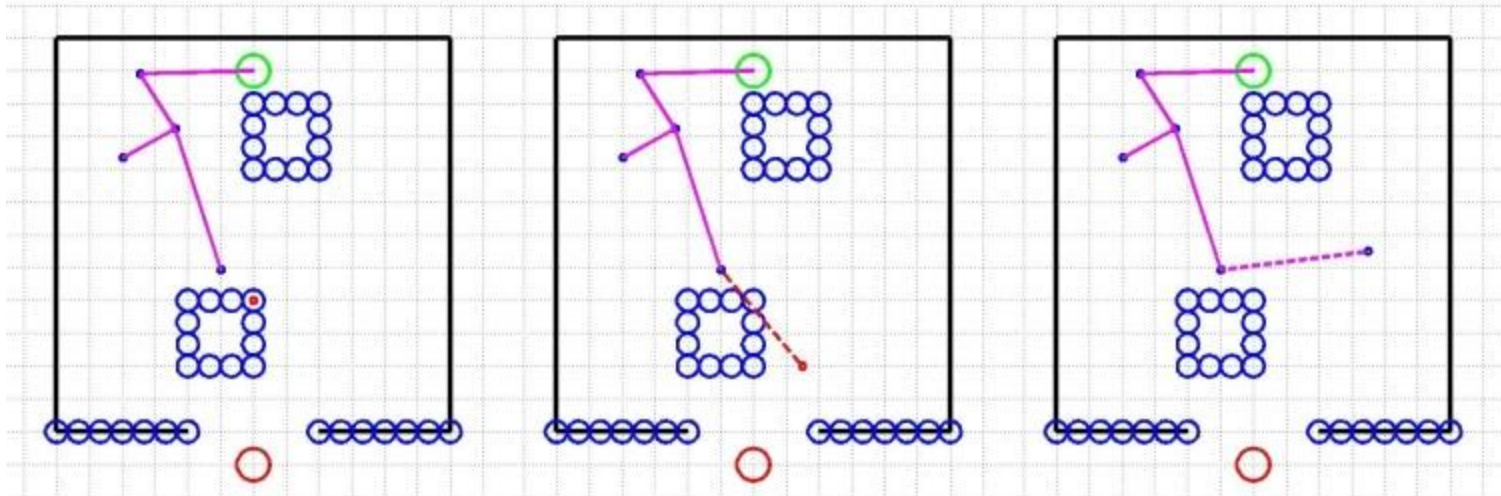




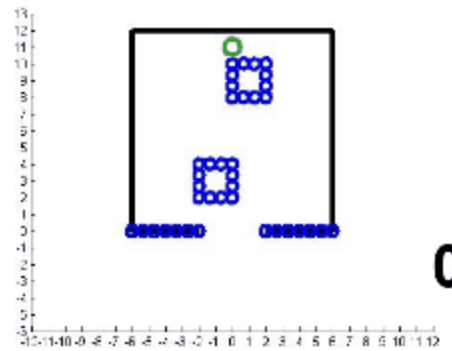
RRT

- RRT Iteration

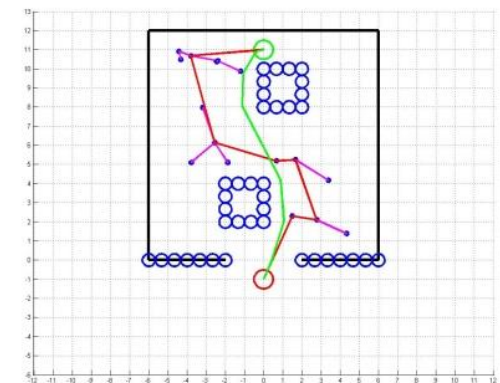
Initial Steps



Final Solution



Iterations



Smoothing





RRT Improvements

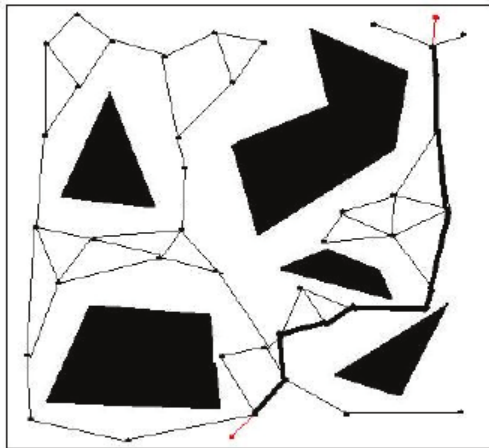
- path shortening
- restricting the area of node sampling (time)
- Grow two trees from origin and goal
- RRT with constraints (nonholonomic)



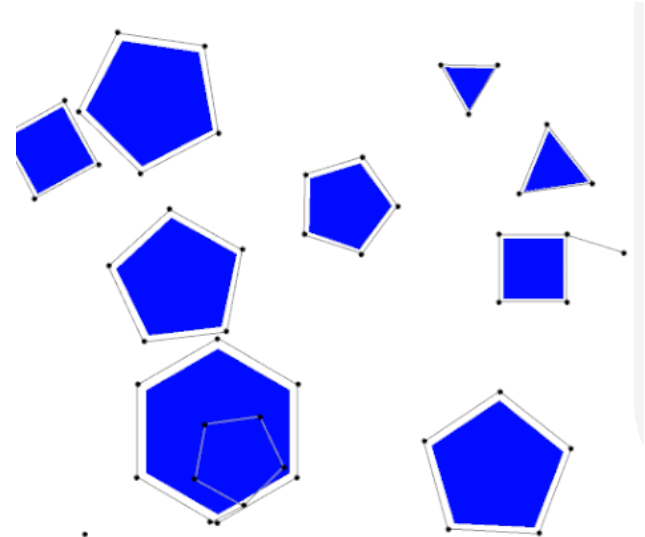


Probabilistic RoadMap Planning

- Probabilistic road-maps create a tree by randomly sampling points in the state-space (pick the points randomly), testing whether they are collision-free, connecting them with neighboring points using paths that reflect the kinematics of a robot, and then using classical graph shortest path algorithms to find shortest paths on the resulting structure.



PRM nodes are generated randomly in the free space, and its edges are drawn such that they lie entirely in the free space.





Global and Local Planners

- The **global planner** provides an overarching path based on the global costmap.
- The **local planner** adjusts this path dynamically, ensuring the robot avoids unforeseen obstacles and follows the planned route safely.





Costmap

- **Global Costmap**

- **Purpose:** Represents the entire known environment.
- **Used by:** The **global planner** for long-term, high-level path planning.
- **Functionality:**
 - Creates an optimal path from the robot's current position to the goal.
 - Incorporates static obstacles and global constraints.

- **Local Costmap**

- **Purpose:** Focuses on the robot's immediate surroundings.
- **Used by:** The **local planner** for short-term, real-time navigation.
- **Functionality:**
 - Handles dynamic obstacles and adjusts the path locally.
 - Prioritizes safety and smooth motion near the robot.





Path Planning in ROS

- Navigation Stack in ROS
 - <http://wiki.ros.org/navigation>
- **global_planner** - Dijkstra's & A* Algorithm
 - http://wiki.ros.org/global_planner
- **rrt_exploration**
 - wiki.ros.org/rrt_exploration
- Open Motion Planning Library - PRM, RRT
 - <https://github.com/ros-gbp/ompl-release>

